Java Code

```java
//1. Write a program to take a list of 10 farmers, fruits they sell, their cities & the quantity of fruits they have as input. Save the data to notepad before & after sorting based on quantity in tabular manner. Also perform search operation based on farmers' name.

import java.util.*;
import java.io.*;

// Class to store farmer details
class Farmer {
    String name;
    String fruit;
    String city;
    int quantity;

    Farmer(String name, String fruit, String city, int quantity) {
        this.name = name;
        this.fruit = fruit;
        this.city = city;
        this.quantity = quantity;
    }
}

public class FarmerDataProgram {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Farmer[] farmers = new Farmer[10];

        // INPUT DATA
        System.out.println("Enter details of 10 farmers:");
        for (int i = 0; i < 10; i++) {
            System.out.println("\nFarmer " + (i + 1));

            System.out.print("Enter Name: ");
            String name = sc.nextLine();

            System.out.print("Enter Fruit: ");
            String fruit = sc.nextLine();

            System.out.print("Enter City: ");
            String city = sc.nextLine();

            System.out.print("Enter Quantity: ");
            int quantity = sc.nextInt();
            sc.nextLine(); // clear buffer

            farmers[i] = new Farmer(name, fruit, city, quantity);
        }

        // SAVE BEFORE SORTING
```

```java
        writeToFile(farmers, "before_sort.txt");

        // SORT BY QUANTITY (Ascending)
        Arrays.sort(farmers, (a, b) -> a.quantity - b.quantity);

        // SAVE AFTER SORTING
        writeToFile(farmers, "after_sort.txt");

        // SEARCH BY NAME
        System.out.print("\nEnter farmer name to search: ");
        String searchName = sc.nextLine();

        boolean found = false;
        for (Farmer f : farmers) {
            if (f.name.equalsIgnoreCase(searchName)) {
                System.out.println("\nFarmer Found:");
                System.out.println("Name: " + f.name);
                System.out.println("Fruit: " + f.fruit);
                System.out.println("City: " + f.city);
                System.out.println("Quantity: " + f.quantity);
                found = true;
                break;
            }
        }

        if (!found) {
            System.out.println("Farmer not found!");
        }

        sc.close();
    }

    // METHOD TO WRITE DATA TO FILE
    public static void writeToFile(Farmer[] farmers, String fileName) {
        try {
            BufferedWriter bw = new BufferedWriter(new FileWriter(fileName));

            // Table Header
            bw.write("Name\tFruit\tCity\tQuantity\n");
            bw.write("-------------------------------------\n");

            // Data
            for (Farmer f : farmers) {
                bw.write(f.name + "\t" + f.fruit + "\t" + f.city + "\t" + f.quantity + "\n");
            }

            bw.close();
            System.out.println("Data saved in " + fileName);

        } catch (IOException e) {
            System.out.println("Error writing file!");
        }
    }
```

```
}

//OUTPUT
Enter details of 10 farmers:

Farmer 1
Enter Name: abhinav
Enter Fruit: apple
Enter City: agra
Enter Quantity: 47

Farmer 2
Enter Name: pooja
Enter Fruit: papaya
Enter City: pune
Enter Quantity: 98

Farmer 3
Enter Name: kishore
Enter Fruit: chikoo
Enter City: calcutta
Enter Quantity: 73

Farmer 4
Enter Name: mahesh
Enter Fruit: lime
Enter City: ladhak
Enter Quantity: 94

Farmer 5
Enter Name: ramesh
Enter Fruit: raspberry
Enter City: bhopal
Enter Quantity: 48

Farmer 6
Enter Name: cemal
Enter Fruit: banana
Enter City: indore
Enter Quantity: 93

Farmer 7
Enter Name: aatish
Enter Fruit: grapes
Enter City: guwahti
Enter Quantity: 73

Farmer 8
Enter Name: samay
Enter Fruit: pomegranate
Enter City: itanagar
Enter Quantity: 66

Farmer 9
```

```
Enter Name: chetak
Enter Fruit: custurd apple
Enter City: himachal
Enter Quantity: 84

Farmer 10
Enter Name: bhavya
Enter Fruit: guava
Enter City: mysore
Enter Quantity: 52
Data saved in before_sort.txt
Data saved in after_sort.txt

Enter farmer name to search: aatish

Farmer Found:
Name: aatish
Fruit: grapes
City: guwahti
Quantity: 73


//2. Write a program to take quantity & quality of fruits Mango, Apple & Papaya as input. Quality may be good, bad or average. If entered quantity of fruit is greater than 1000, store it as half. Perform search for a given fruit with given quality.
import java.util.*;

class Fruit {
    String name;
    int quantity;
    String quality;

    Fruit(String name, int quantity, String quality) {
        this.name = name;

        // If quantity > 1000, store half
        if (quantity > 1000) {
            this.quantity = quantity / 2;
        } else {
            this.quantity = quantity;
        }

        this.quality = quality;
    }
}

public class FruitQualitySearch {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        Fruit[] fruits = new Fruit[3];
        String[] fruitNames = {"Mango", "Apple", "Papaya"};

        // INPUT
```

```java
        for (int i = 0; i < 3; i++) {
            System.out.println("\nEnter details for " + fruitNames[i]);

            System.out.print("Enter Quantity: ");
            int qty = sc.nextInt();
            sc.nextLine(); // clear buffer

            System.out.print("Enter Quality (good/bad/average): ");
            String quality = sc.nextLine();

            fruits[i] = new Fruit(fruitNames[i], qty, quality);
        }

        // SEARCH
        System.out.print("\nEnter fruit name to search: ");
        String searchFruit = sc.nextLine();

        System.out.print("Enter quality to search: ");
        String searchQuality = sc.nextLine();

        boolean found = false;

        for (Fruit f : fruits) {
            if (f.name.equalsIgnoreCase(searchFruit) &&
                f.quality.equalsIgnoreCase(searchQuality)) {

                System.out.println("\nFruit Found:");
                System.out.println("Name: " + f.name);
                System.out.println("Quantity: " + f.quantity);
                System.out.println("Quality: " + f.quality);

                found = true;
                break;
            }
        }

        if (!found) {
            System.out.println("Fruit not found!");
        }

        sc.close();
    }
}

//OUTPUT
Enter details for Mango
Enter Quantity: 1200
Enter Quality (good/bad/average): good

Enter details for Apple
Enter Quantity: 800
Enter Quality (good/bad/average): average

Enter details for Papaya
```

```
Enter Quantity: 1500
Enter Quality (good/bad/average): bad

Enter fruit name to search: Mango
Enter quality to search: good

Fruit Found:
Name: Mango
Quantity: 600
Quality: good


// 3. There are four orchards, namely A, B, C & D with fruits in them as Apple, Litchi, Mango, Apple respectively. The qualities of the fruits in the orchards are good, good, good & bad respectively.The available orchards in a tree-view.Search for a given fruit in the available orchards after taking input from the user & print the tree.
import java.util.Scanner;

class Orchard {
    String name, fruit, quality;
    Orchard left, right;

    Orchard(String name, String fruit, String quality) {
        this.name = name;
        this.fruit = fruit;
        this.quality = quality;
        left = right = null;
    }
}

public class OrchardTree {

    // Print tree (Inorder)
    static void printTree(Orchard root) {
        if (root != null) {
            printTree(root.left);
            System.out.println(root.name + " -> " + root.fruit + " (" + root.quality + ")");
            printTree(root.right);
        }
    }

    // Search fruit
    static void searchFruit(Orchard root, String key) {
        if (root != null) {
            searchFruit(root.left, key);

            if (root.fruit.equalsIgnoreCase(key)) {
                System.out.println("Found in Orchard " + root.name +
                                   " with quality: " + root.quality);
            }

            searchFruit(root.right, key);
```

```java
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Create orchards (Tree structure)
        Orchard A = new Orchard("A", "Apple", "good");
        Orchard B = new Orchard("B", "Litchi", "good");
        Orchard C = new Orchard("C", "Mango", "good");
        Orchard D = new Orchard("D", "Apple", "bad");

        // Making tree
        A.left = B;
        A.right = C;
        B.left = D;

        // Input
        System.out.print("Enter fruit to search: ");
        String fruit = sc.nextLine();

        // Search
        searchFruit(A, fruit);

        // Print Tree
        System.out.println("\nTree View:");
        printTree(A);
    }
}

//OUTPUT
Enter fruit to search: Apple

Found in Orchard D with quality: bad
Found in Orchard A with quality: good

Tree View:
D -> Apple (bad)
B -> Litchi (good)
A -> Apple (good)
C -> Mango (good)


//Take a map provided as a graph & display all the subtrees of connected points.
import java.util.*;

public class GraphSubtrees {

    // DFS to find connected components
    static void dfs(int node, boolean[] visited, ArrayList<ArrayList<Integer>> graph)
{
        visited[node] = true;
        System.out.print(node + " ");

```

```java
        for (int neighbor : graph.get(node)) {
            if (!visited[neighbor]) {
                dfs(neighbor, visited, graph);
            }
        }
    }

    public static void main(String[] args) {
        int n = 6; // number of nodes (0 to 5)

        // Graph (Adjacency List)
        ArrayList<ArrayList<Integer>> graph = new ArrayList<>();

        for (int i = 0; i < n; i++) {
            graph.add(new ArrayList<>());
        }

        // Adding edges (map connections)
        graph.get(0).add(1);
        graph.get(1).add(0);

        graph.get(1).add(2);
        graph.get(2).add(1);

        graph.get(3).add(4);
        graph.get(4).add(3);

        // Node 5 is separate

        boolean[] visited = new boolean[n];

        // Find all subtrees (connected components)
        System.out.println("Subtrees (Connected Components):");

        for (int i = 0; i < n; i++) {
            if (!visited[i]) {
                System.out.print("Component: ");
                dfs(i, visited, graph);
                System.out.println();
            }
        }
    }
}

//OUTPUT
Subtrees (Connected Components):
Component: 0 1 2
Component: 3 4
Component: 5


//5. Enter entry point & goal point & produce the matching subtree of the graph.
import java.util.*;

```

```java
public class GraphPath {

    static boolean dfs(int node, int goal, boolean[] visited,
                       ArrayList<ArrayList<Integer>> graph,
                       ArrayList<Integer> path) {

        visited[node] = true;
        path.add(node);

        // If goal found
        if (node == goal) {
            return true;
        }

        // Visit neighbors
        for (int neighbor : graph.get(node)) {
            if (!visited[neighbor]) {
                if (dfs(neighbor, goal, visited, graph, path)) {
                    return true;
                }
            }
        }

        // Backtrack (remove if not correct path)
        path.remove(path.size() - 1);
        return false;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = 6;

        ArrayList<ArrayList<Integer>> graph = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            graph.add(new ArrayList<>());
        }

        // Graph connections
        graph.get(0).add(1);
        graph.get(1).add(0);

        graph.get(1).add(2);
        graph.get(2).add(1);

        graph.get(3).add(4);
        graph.get(4).add(3);

        graph.get(2).add(5);
        graph.get(5).add(2);

        System.out.print("Enter Entry Point: ");
        int start = sc.nextInt();

```

```java
        System.out.print("Enter Goal Point: ");
        int goal = sc.nextInt();

        boolean[] visited = new boolean[n];
        ArrayList<Integer> path = new ArrayList<>();

        if (dfs(start, goal, visited, graph, path)) {
            System.out.println("Matching Subtree (Path): " + path);
        } else {
            System.out.println("No path found");
        }
    }
}

//OUTPUT
Enter Entry Point: 0
Enter Goal Point: 5

Matching Subtree (Path): [0, 1, 2, 5]


//6. Give the weightage for each edge of nodes, and produce the shortest path from start to end node.
import java.util.*;

class Pair {
    int node, weight;
    Pair(int n, int w) {
        node = n;
        weight = w;
    }
}

public class ShortestPath {

    static void dijkstra(int start, int end, ArrayList<ArrayList<Pair>> graph, int n) {

        int[] dist = new int[n];
        int[] parent = new int[n];

        Arrays.fill(dist, Integer.MAX_VALUE);

        PriorityQueue<Pair> pq = new PriorityQueue<>((a, b) -> a.weight - b.weight);

        dist[start] = 0;
        parent[start] = -1;

        pq.add(new Pair(start, 0));

        while (!pq.isEmpty()) {
            Pair curr = pq.poll();

            for (Pair neighbor : graph.get(curr.node)) {
```

```java
                int newDist = dist[curr.node] + neighbor.weight;

                if (newDist < dist[neighbor.node]) {
                    dist[neighbor.node] = newDist;
                    parent[neighbor.node] = curr.node;
                    pq.add(new Pair(neighbor.node, newDist));
                }
            }
        }

        // Print shortest distance
        System.out.println("Shortest Distance: " + dist[end]);

        // Print path
        ArrayList<Integer> path = new ArrayList<>();
        for (int v = end; v != -1; v = parent[v]) {
            path.add(v);
        }

        Collections.reverse(path);
        System.out.println("Shortest Path: " + path);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = 5;

        ArrayList<ArrayList<Pair>> graph = new ArrayList<>();

        for (int i = 0; i < n; i++) {
            graph.add(new ArrayList<>());
        }

        // Adding weighted edges
        graph.get(0).add(new Pair(1, 2));
        graph.get(1).add(new Pair(0, 2));

        graph.get(1).add(new Pair(2, 3));
        graph.get(2).add(new Pair(1, 3));

        graph.get(0).add(new Pair(3, 6));
        graph.get(3).add(new Pair(0, 6));

        graph.get(3).add(new Pair(4, 1));
        graph.get(4).add(new Pair(3, 1));

        graph.get(2).add(new Pair(4, 5));
        graph.get(4).add(new Pair(2, 5));

        System.out.print("Enter Start Node: ");
        int start = sc.nextInt();

        System.out.print("Enter End Node: ");
```

```java
        int end = sc.nextInt();

        dijkstra(start, end, graph, n);
    }
}

//OUTPUT
Enter Start Node: 0
Enter End Node: 4

Shortest Distance: 7
Shortest Path: [0, 3, 4]


//7.Write a program to demonstrate the use of all keywords made for exception handling.
import java.util.Scanner;

class Demo {

    // using throws
    static void checkAge(int age) throws Exception {
        if (age < 18) {
            // using throw
            throw new Exception("Not eligible to vote");
        } else {
            System.out.println("Eligible to vote");
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try {
            System.out.print("Enter a number: ");
            int a = sc.nextInt();

            System.out.print("Enter another number: ");
            int b = sc.nextInt();

            int result = a / b;   // may cause ArithmeticException
            System.out.println("Result: " + result);

            System.out.print("Enter age: ");
            int age = sc.nextInt();

            checkAge(age);   // calling method with throws

        }
        catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero");
        }
        catch (Exception e) {
            System.out.println("Exception: " + e.getMessage());
```

```java
        }
        finally {
            System.out.println("Program finished (finally block)");
        }
    }
}

//OUTPUT
Enter a number: 10
Enter another number: 0
Cannot divide by zero
Program finished (finally block)


//8. Write a program to withraw money from a bank account, if the withdrawn money is greater than the balance, then it throws NoMoneyException.
import java.util.Scanner;

// Custom Exception
class NoMoneyException extends Exception {
    NoMoneyException(String msg) {
        super(msg);
    }
}

public class BankWithdraw {

    // Method for withdrawal
    static void withdraw(int balance, int amount) throws NoMoneyException {
        if (amount > balance) {
            throw new NoMoneyException("Insufficient Balance!");
        } else {
            balance = balance - amount;
            System.out.println("Withdrawal Successful");
            System.out.println("Remaining Balance: " + balance);
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int balance = 5000;

        System.out.println("Current Balance: " + balance);
        System.out.print("Enter amount to withdraw: ");
        int amount = sc.nextInt();

        try {
            withdraw(balance, amount);
        }
        catch (NoMoneyException e) {
            System.out.println("Exception: " + e.getMessage());
        }
        finally {
```

```
            System.out.println("Transaction Ended");
        }
    }
}

//OUTPUT
Current Balance: 5000
Enter amount to withdraw: 4789
Withdrawal Successful
Remaining Balance: 211
Transaction Ended

//9. Write a program which enables to use a platform by a train one at a time.
class Platform {

    // synchronized method (only one train at a time)
    synchronized void usePlatform(String trainName) {
        System.out.println(trainName + " is using the platform");

        try {
            Thread.sleep(2000); // simulate time
        } catch (Exception e) {}

        System.out.println(trainName + " has left the platform");
    }
}

// Train class (Thread)
class Train extends Thread {
    Platform p;
    String name;

    Train(Platform p, String name) {
        this.p = p;
        this.name = name;
    }

    public void run() {
        p.usePlatform(name);
    }
}

public class Railway {
    public static void main(String[] args) {

        Platform p = new Platform();

        Train t1 = new Train(p, "Train 1");
        Train t2 = new Train(p, "Train 2");
        Train t3 = new Train(p, "Train 3");

        t1.start();
        t2.start();
        t3.start();
```

```
    }
}

//OUTPUT
Train 2 is using the platform
Train 2 has left the platform
Train 3 is using the platform
Train 3 has left the platform
Train 1 is using the platform
Train 1 has left the platform


//10. Write a programme of Restaurant and Waiter, which accept order if kitchen if free and sheff deliver orders if he is free
class Kitchen {
    boolean orderReady = false;

    // Waiter places order
    synchronized void placeOrder(String order) {
        try {
            if (orderReady) {
                wait(); // wait if kitchen busy
            }

            System.out.println("Waiter placed order: " + order);
            orderReady = true;

            notify(); // inform chef

        } catch (Exception e) {}
    }

    // Chef prepares order
    synchronized void prepareOrder() {
        try {
            if (!orderReady) {
                wait(); // wait if no order
            }

            System.out.println("Chef prepared the order");
            orderReady = false;

            notify(); // inform waiter

        } catch (Exception e) {}
    }
}

// Waiter Thread
class Waiter extends Thread {
    Kitchen k;

    Waiter(Kitchen k) {
        this.k = k;
```

```java
    }

    public void run() {
        String[] orders = {"Pizza", "Burger", "Pasta"};

        for (String o : orders) {
            k.placeOrder(o);
        }
    }
}

// Chef Thread
class Chef extends Thread {
    Kitchen k;

    Chef(Kitchen k) {
        this.k = k;
    }

    public void run() {
        for (int i = 0; i < 3; i++) {
            k.prepareOrder();
        }
    }
}

public class Restaurant {
    public static void main(String[] args) {

        Kitchen k = new Kitchen();

        Waiter w = new Waiter(k);
        Chef c = new Chef(k);

        w.start();
        c.start();
    }
}

//OUTPUT
Waiter placed order: Pizza
Chef prepared the order
Waiter placed order: Burger
Chef prepared the order
Waiter placed order: Pasta
Chef prepared the order
```